Homework 1: Machine numbers; Interpolation

Due Oct. 13th.

1. Floating point representation

   For the `float` data type, write a program to **empirically** (i.e., by performing tests on the results of addition and subtraction operations within your program) determine the following "Machine constants" for your computer:

   (a) The smallest ϵ such that $1.0 - \epsilon \neq 1.0$

   (b) The smallest ϵ such that $1.0 + \epsilon \neq 1.0$

   (c) The maximum representable number

   (d) The minimum representable positive number

   Provide all the above answers to better than 10% accuracy, and comment on why the numbers you get are expected based on the IEEE 754 representation.

2. Floating point representation and rounding

   Write a method to sum the elements of an array using using only `numpy.float32` operations (32 bit IEEE 754 floating point). For this part of the problem, avoid using the `numpy` internal sum method (see below). Generate an array of 20 million random numbers of type `float32` (32 bit IEEE 754 floating point) between 0 and 1 (use an available method such as `numpy.random.rand()` in Python), and use your method to calculate the the sum of the numbers in the array

   (a) in the order as produced by `rand()`,

   (b) after being sorted from smallest to largest

   (c) after being sorted from largest to smallest.

   Explain the differences in the results from your `sum` method based on the limitations of the IEEE 754 precision. Also calculate the above sums using the internal `numpy.ndarray.sum()` method. Speculate on how the internal method differs from your implementation.

3. Roundoff error

   Numerically evaluate the expression $(1-\cos(x))/x^2$ in double precision for values of x around 10^{-7} and smaller. Explain the difference between the numerical results and the analytic limit as $x \to 0$.

   Explain why trouble is occurring at $x \sim 10^{-7}$ when double precision is good to $\sim 10^{-15}$?

4. Interpolation

(a) Write a program to read in a two column table from a file and perform linear interpolation at an arbitrary point. You may assume that the data is evenly spaced in the independent variable (this makes it easier to determine which points to use for interpolation).

(b) Use the program on the following input data: (available on the canvas site as `hw_data.csv`)

x	y
1	100
2	25
3	11.111111
4	6.25
5	4

and provide an estimate of y at $x = 4.75$.

(c) Either using your own routine or a package (e.g. `scipy.interpolate`) calculate interpolated values of y at 100 equally spaced points between $x = 1$ and $x = 5$ using the above data with linear, quadratic, and 4th order polynomials, and plot the interpolated y vs. x. (Note: we are NOT performing a "fit" here; we want polynomials that go exactly through all the given data points.) Give an estimate of y at $x = 4.75$ for each of these polynomial orders.

(d) The actual function tabulated above is $y = \frac{100}{x^2}$. Compare the actual value at $x = 4.75$ with the linear interpolation and the 4th order polynomial interpolation, and comment on why one is more accurate than the other.